

MedLedger

Trust the Math, Not the Admin

Blockchain-Powered Patient-Centric Healthcare Data Management

TEAM

Team Name: Praxis

Name	Role	Contribution
Mayanglambam Premananda Singh	Blockchain Architect	Cryptography & consensus design
Mayanglambam Premananda Singh	Backend Developer	FastAPI, crypto ops, API architecture
Korounganba Thochon & Thajaba Naoroibam	Frontend Developer	React 18, patient-facing UX
[Korounganba Thochon & Thajaba Naoroibam	Healthcare Expert	Compliance, domain validation, HIPAA

Submission Links:

- [Round 1 Concept Video:](https://drive.google.com/drive/folders/1E2nfH2up5BBV9a7kZd_50SgWbdqm3JKW?usp=drive_link) https://drive.google.com/drive/folders/1E2nfH2up5BBV9a7kZd_50SgWbdqm3JKW?usp=drive_link
- Round 2 Demo Video: Coming Soon
- GitHub Repo: <https://github.com/premananda-cloud/MedLedger>

1. THE PROBLEM LANDSCAPE

Problem Statement

Healthcare data sits behind admin-controlled systems where insiders can silently view, modify, or delete access logs. 58% of healthcare breaches involve internal actors (Verizon 2024) — yet patients have no visibility, no control, and no recourse. Audit logs live in the same database as the records, meaning a single breach can erase both data and evidence.

Target Audience

- Patients — who deserve control over their own sensitive records.
- Hospitals — facing \$10.1M average breach costs and escalating HIPAA fines (now up to \$5.1M per violation).
- Regulators — who currently must trust hospitals' self-reported audit logs.

Why Now?

66% of healthcare organisations were hit by ransomware in 2023. 73% of patients want data control they don't have. Technology readiness has arrived: ECDSA, AES-256-GCM, and Shamir's Secret Sharing are proven, library-ready primitives. The gap is not capability — it's implementation.

2. PROPOSED SOLUTION & USP

Solution in One Sentence

Patient signs access permission with a private key — hospital admin cannot override, every access is logged forever on-chain, and logs cannot be deleted.

How It Works

- Patient Registration: ECDSA P-256 keypair generated. Private key stays with the patient only; public key stored on blockchain.
- Access Grant: Doctor requests record. Patient approves with a time-limited, cryptographically-signed permission (e.g. "Dr. Smith, today 2 pm-4 pm only"). Signature committed to blockchain — immutable.
- Access Enforcement: At access time the system verifies the on-chain signature and time window. Outside the window, or without a valid grant, access is mathematically impossible — not just policy-blocked.
- Audit Trail: Every event (grant, access, denial, revoke) is hash-chained to the previous entry and committed to blockchain. Tampering breaks the chain and is instantly detectable.

Unique Selling Proposition

Current systems rely on admin goodwill and deletable logs. MedLedger replaces trust with math.

Feature	Epic/Cerner	Trad. Blockchain	MedLedger
Admin secret access	✗ Possible	✗ Possible	✓ Math-blocked
Deletable audit logs	✗ Deletable	✗ Deletable	✓ Blockchain-locked
Patient access revoke	☐ Slow/manual	✗ Limited	✓ 1-click instant
Time-limited access	✗ No	☐ Custom build	✓ Built-in, enforced
Key loss recovery	N/A	✗ Lost forever	✓ 3-of-5 Shamir
Performance	✓ Fast	✗ Slow on-chain	✓ Hybrid (fast)

3. TECHNICAL ARCHITECTURE & STACK

System Workflow

- Input: Patient uploads medical record → encrypted client-side with AES-256-GCM → stored on AWS S3 (ciphertext only). Record hash + metadata committed to blockchain.
- Processing: Permission requests trigger patient signature flow. API verifies on-chain grants before decrypting and serving records. Every verified (and denied) access event is signed and hash-chained.
- Output: Authorised viewer receives decrypted record. Blockchain retains a tamper-proof, independently verifiable audit log for regulators, patients, and compliance teams.

Tech Stack

Layer	Technology	Rationale
Frontend	React 18 + TypeScript	Fast, type-safe patient UI
Backend / API	FastAPI (Python)	Async, ideal for cryptographic operations
Cryptography	cryptography.io (ECDSA, AES-256-GCM)	NIST-standard proven libraries
Database	PostgreSQL	ACID-compliant metadata & access permissions
Storage	AWS S3 (AES-256 encrypted)	Encrypted record blobs, unlimited scale
Blockchain	Custom consensus layer	Hash-chained audit log with PBFT-inspired signing
Deployment	Docker + Railway/Heroku	Ship in days, not weeks

Architecture (Simplified)

Patient App (React) → FastAPI Server (Crypto Ops + Permission Check) → Three stores in parallel:

- PostgreSQL – metadata, user records, hashed permissions.
- AWS S3 – AES-256 encrypted record blobs (zero plaintext leaves the server).
- Blockchain Layer – immutable hash-chained audit events; PBFT-inspired multi-party signing for consensus.

4. KEY FEATURES & FUNCTIONALITIES

Feature 1 — Cryptographic Access Control (Core MVP)

Patient grants time-windowed, record-specific access via a private-key signature. Math enforces the window — no admin override is architecturally possible. Failed access attempts are also logged on-chain, providing an irrefutable breach-attempt record.

Feature 2 — Immutable Audit Trail (UX / Compliance)

Every event forms a hash chain where each entry's hash is embedded in the next. The patient dashboard displays a real-time, human-readable audit log. Regulators can independently verify chain integrity without trusting the hospital.

Feature 3 — Shamir Key Recovery (Reliability / Safety Net)

If a patient loses their private key, a 3-of-5 Shamir Secret Sharing scheme distributes recovery fragments across five independent locations (secure vault, hospital, blockchain, email, trusted contact). Recovery completes in 30 minutes without centralising custody.

5. IMPLEMENTATION ROADMAP

Timeline	Phase	Deliverable
Days 1-2	Phase 1	FastAPI setup, ECDSA keypair generation
Days 3-4	Phase 1	User registration + JWT auth
Days 5-6	Phase 1	Shamir Secret Sharing (key recovery)
Day 7	Phase 1	Demo: patient grants access, admin denied – Concept Video
Days 8-9	Phase 2	AES-256-GCM record encryption + S3 upload
Day 10	Phase 2	Permission granting UI + blockchain log submission
Day 11	Phase 2	Audit trail hash-chain verification
Days 12-13	Phase 2	UI polish, bug fixes, stress testing
Day 14	Phase 2	Final 2-min demo video + documentation

6. IMPACT & SUSTAINABILITY

Stakeholder	Problem	MedLedger Outcome
Patients	Zero visibility into who accesses their data	Cryptographic control + 1-click revoke + immutable audit
Hospitals	Avg. \$10.1M breach cost; HIPAA audit burden	Tamper-proof logs eliminate audit overhead; breach risk drops
Doctors	Delayed record access; cumbersome auth	Instant access once patient grants; no admin bottleneck
Regulators	Rely on hospitals self-reporting audit logs	Independently verifiable blockchain trail — no trust required

Scalability

- MVP: Single server, ~100 patients, 1 hospital – sufficient for hackathon demo.
- Year 1: PostgreSQL read-replicas, S3, 100K patients across 10 hospitals – no architectural change required.
- Year 5: Sharded DB by patient ID, distributed IPFS + regional S3, multi-region blockchain – linear scale from the same core design.

Risk & Mitigation

Primary Risk: Patient key loss leads to data inaccessibility.

Mitigation: Shamir 3-of-5 recovery built into Phase 1. Post-hackathon path includes post-quantum algorithm swap (CRYSTALS-Dilithium) via crypto-agile architecture — no redesign needed.

7. REFERENCES

- Shamir, A. (1979). How to Share a Secret. Communications of the ACM, 22(11).
- NIST FIPS 186-4 – ECDSA P-256 Digital Signature Standard.
- NIST SP 800-38D – AES-256-GCM Authenticated Encryption.
- Castro & Liskov (1999). Practical Byzantine Fault Tolerance. OSDI.
- HIPAA Audit Controls – 45 CFR § 164.312(b).
- Verizon 2024 Data Breach Investigations Report – Healthcare sector insider threat statistics.
- cryptography.io – ECDSA, AES, PBKDF2 library. <https://cryptography.io>
- FastAPI – <https://fastapi.tiangolo.com>
- Synthea Synthetic Patient Data – <https://www.synthetichealth.org>

"Trust the Math, Not the Admin" — Math does not take bribes. Blockchain does not delete logs.